As you can see, `CreateDefaultBuilder` does a lot for you. In many cases, these defaults are exactly what you need, but if they're not, the default builder is optional. You could call `new HostBuilder()` and start customizing it from there, but you'd need to set up everything that `CreateHostBuilder` does: logging, hosting configuration, and service provider configuration, as well as your app configuration.

An alternative approach is to layer additional configuration on top of the existing defaults. In the following listing, I show how to add a Seq logging provider to the configured providers using `ConfigureLogging()`, as well as how to reconfigure the app configuration to load *only* from the appsettings.json provider by clearing the default providers.

Listing 30.6 Customizing the default `HostBuilder`

```
public class Program
{
    public static void Main(string[] args)
    {
        CreateHostBuilder(args).Build().Run();
    }

    public static IHostBuilder CreateHostBuilder(string[] args) =>
        Host.CreateDefaultBuilder(args)
            .ConfigureLogging(logBuilder => logBuilder.AddSeq())  ❶
            .ConfigureAppConfiguration((hostContext, config) =>   ❷
            {
             config.Sources.Clear();                              ❸
                    config.AddJsonFile("appsettings.json");       ❹

            }
            .ConfigureWebHostDefaults(webBuilder =>
            {
                webBuilder.UseStartup<Startup>();
            });
}
```

❶ Adds the Seq logging provider to the configuration

❷ HostBuilder provides a hosting context and an instance of ConfigurationBuilder.